

Security Audit

Defi Bull World 2nd (Defi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	12
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Defi Bull World team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Defi Bull World is a cutting-edge digital asset and DeFi education platform built on the Flare Network with its native token FLR, led by seasoned blockchain experts since Bitcoin's inception. The platform delivers expert insights and regular updates through courses, webinars, and multimedia content, with plans to go multichain by Q3 2025. With new courses added monthly, AI-powered multilingual content, interactive experiences like sweepstakes and tests, and guaranteed long-term access for early members, Defi Bull World is committed to transparency through regular audits and public disclosure of contracts and wallets, while reinvesting revenues to drive sustainable growth.

Project Name: Defi Bull World

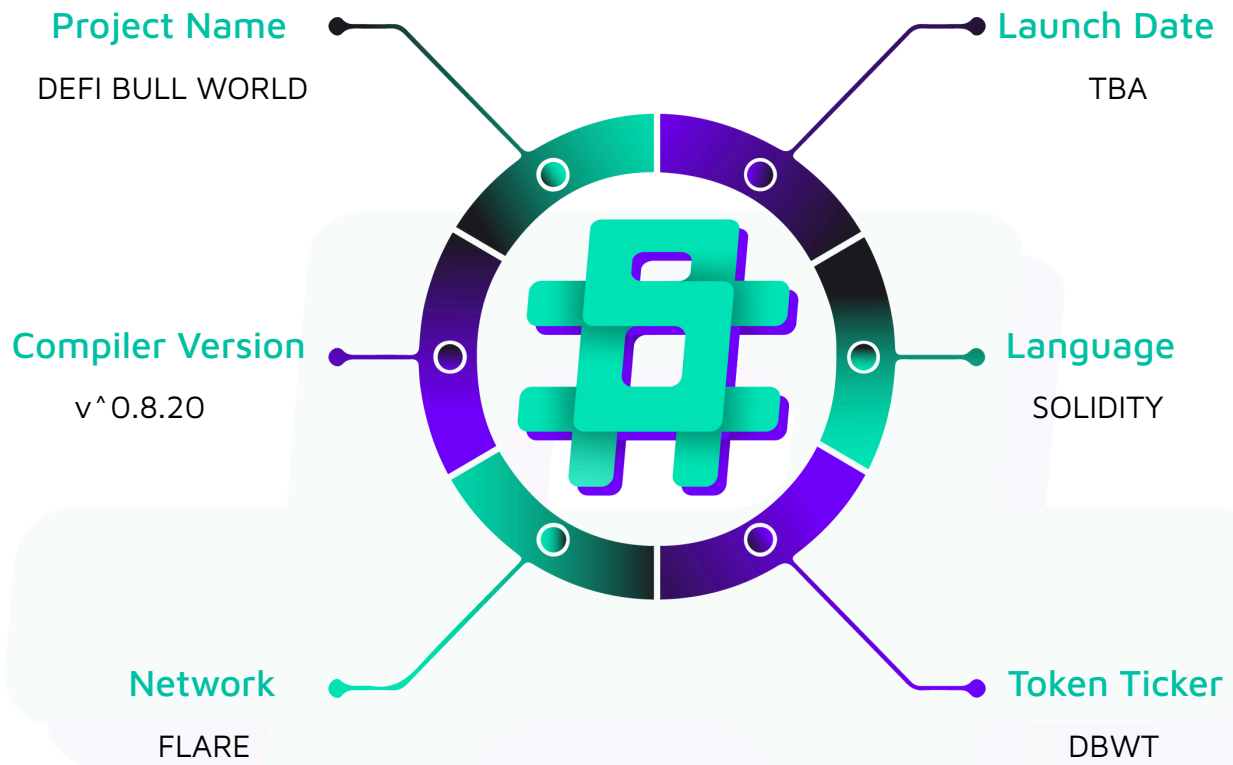
Project Type: DeFi

Compiler Version: ^0.8.20

Website: <https://www.defibullworld.com/>

Logo:



Visualised Context:

Project Visuals:

Master DeFi Education Unlock Your Financial Future

Join the revolution in decentralized finance education. Learn, earn, and grow.

[Start Learning & Earning →](#)

[View Membership](#)



Who We Are

We're not just an education platform – we're a movement dedicated to redefining how decentralized finance is learned and experienced.

Founded on the belief that knowledge is the foundation of empowerment, our platform bridges the gap between traditional finance and the new digital economy. With a team of industry experts and cutting-edge technology, we're committed to delivering a secure, transparent, and collaborative environment where everyone can thrive in the DeFi space.



Transparent Education

We deliver in-depth DeFi insights with clarity and integrity, empowering you with actionable knowledge.



Robust Security

Experience top-tier protection with state-of-the-art protocols that secure your digital journey.



Community Empowerment

Become part of a global network that values collaboration, support, and shared growth.



Data-Driven Insights

Leverage real-time market analysis and trends curated by industry experts to make informed decisions.

Audit Scope

We at Hashlock audited the solidity code within the Defi Bull World project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Defi Bull World Smart Contracts
Platform	Flare / Solidity
Audit Date	May, 2025
Contract 1	MembershipPaymentManager.sol
Contract 1 MD5 Hash	adb9ab923730f074b13a91107cb32935

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

5 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
MembershipPaymentManager.sol Allows users to: - Purchase tiered memberships with multiple currencies - Earn referral commissions through multi-level system - Claim pending rewards Allows admins to: - Configure pricing and commission rates - Manage protocol wallets - Pause/unpause contract functions	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Defi Bull World project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Defi Bull World project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] MembershipPaymentManager#_claimERC20Rewards - Depositors can lose all their deposited tokens if their address is blacklisted

Description

The `claimERC20Rewards` function in `MembershipPaymentManager` sends tokens strictly to `msg.sender` without allowing the caller to specify a withdrawal destination address. If a user's address is blacklisted by USDC/USDT, their funds/rewards will become permanently locked in the contract with no recovery method.

Vulnerability Details

The current implementation of the `claimERC20Rewards` function in `MembershipPaymentManager` automatically sends tokens to the caller's address:

```
// New function to claim ERC20 token referral rewards (pull payment)

function claimERC20Rewards(
    address tokenAddress
) external nonReentrant whenNotPaused returns (uint256) {
    uint256 amount = pendingERC20Rewards[msg.sender][tokenAddress];

    require(amount > 0, "No rewards to claim for this token");

    // Reset pending rewards before transfer to prevent reentrancy
    pendingERC20Rewards[msg.sender][tokenAddress] = 0;

    // Transfer rewards
    IERC20 token = IERC20(tokenAddress);

    require(token.transfer(msg.sender, amount), "Reward transfer failed");

    emit ReferralRewardsClaimed(msg.sender, amount, block.timestamp);
```

```
return amount;  
  
}
```

The underlying `claimERC20Rewards` function transfers tokens to `msg.sender` without providing any option to specify an alternative recipient. USDC and USDT has an active blacklisting mechanism that can prevent transfers to specific addresses. If a user becomes blacklisted, any attempt to withdraw will fail as the USDC/USDT transfer operation will revert. Since there's no way to specify an alternative withdrawal address, the user's funds become permanently locked in the contract..

Impact

This vulnerability can lead to permanent loss of user rewards if they become blacklisted by USDC/USDT. The issue is particularly concerning because blacklisting can happen for various regulatory reasons beyond the user's control, and once it occurs, there is no recovery mechanism within the protocol to retrieve the locked funds.

Recommendation

Modify the `claimERC20Rewards()` function to accept an optional recipient address parameter.

Status

Resolved

[M-02] MembershipPaymentManager#processPayment - Fixed Slippage Parameter Prevents Transactions During Market Volatility

Description

The `MembershipPaymentManager` contract uses a hardcoded slippage parameter (default 5%) that can only be adjusted by the contract owner. During periods of high market volatility, this fixed slippage could cause all membership purchase transactions to fail, temporarily denying service to the user.

Vulnerability Details

In the `processPayment` function, it calls the `calculateAndValidatePayment()`:

```
function calculateAndValidatePayment(
    uint256 tier,
    uint256 amount,
    uint256 paymentValue
) public view returns (bool) {
    uint256 usdPrice = tierUSDPrices[tier] * amount;
    uint256 currentFlareRequired = getFlareAmount(usdPrice);

    // Calculate min/max acceptable payment with slippage tolerance
    uint256 minAcceptable = (currentFlareRequired *
        (100 - slippagePercentage)) / 100;
    uint256 maxAcceptable = (currentFlareRequired *
        (100 + slippagePercentage)) / 100;

    // Check if payment is within acceptable range
    return (paymentValue >= minAcceptable && paymentValue <= maxAcceptable);
}
```

This function validates payment amounts against a slippage tolerance calculated from the current FLR price. The `slippagePercentage` (default 5%, max 10%) is used to create an acceptable price range. This design prevents users from specifying their own acceptable slippage, which can lead to transaction failures during volatile market conditions.

Impact

During periods of high volatility in the token prices, users will be unable to purchase memberships. This creates a denial of service condition for the primary functions of the contract.

Recommendation

Allow users to specify their own maximum slippage parameter.

Status

Resolved

Low

[L-01] Contracts - Use of floating pragma

Description

The contract `MembershipPaymentManager.sol` uses a floating pragma version (`^0.8.20`). It's crucial to compile and deploy contracts with the same compiler version and settings used during development and testing. Fixing the pragma version prevents compilation with different versions. Using an outdated pragma version could introduce bugs that negatively affect the contract system, while newly released versions may have undiscovered security vulnerabilities.

Recommendation

Change the pragma statement in `MembershipPaymentManager.sol` contract to a fixed version, such as `pragma solidity 0.8.20;`. This locks the contract to a specific compiler version.

Status

Resolved

[L-02] Contracts - Unused events in `MembershipPaymentManager.sol`

Description

The `MembershipPaymentManager` contract defines events that are never emitted throughout the codebase. Specifically, `TokensWithdrawn(address token, address recipient, uint256 amount)` is defined but not emitted in the `withdrawERC20` function where it would be appropriate. Additionally, `ContractPaused(address account)` and `ContractUnpaused(address account)` events are redundant since the `PausableUpgradeable` contract that this contract inherits already defines and emits similar events (`Paused` and `Unpaused`) when `_pause()` and `_unpause()` internal functions are called.

Recommendation

The redundant `ContractPaused` and `ContractUnpaused` events should be removed since `PausableUpgradeable` already provides this functionality. For the `TokensWithdrawn` event, either add an emit statement in the `withdrawERC20` function (e.g., `emit TokensWithdrawn(tokenAddress, recipient, withdrawAmount);` before returning success.

Status

Resolved

[L-03] Contracts - Zero Slippage Allowed in `setSlippagePercentage` Function

Description

The `setSlippagePercentage` function lacks validation to prevent setting slippage to 0. Allowing zero slippage can lead to transaction reverts in price calculations or swaps that use this parameter, as most DeFi operations require some minimum slippage tolerance to execute successfully.

Recommendation

Add validation to prevent setting slippage to 0:

```
function setSlippagePercentage(uint256 _newSlippagePercentage) external onlyOwner {
    require(_newSlippagePercentage > 0, "Slippage cannot be zero");
    require(_newSlippagePercentage <= 10, "Slippage cannot exceed 10%");

    uint256 oldPercentage = slippagePercentage;
    slippagePercentage = _newSlippagePercentage;

    emit SlippagePercentageUpdated(oldPercentage, _newSlippagePercentage);
}
```

Status

Resolved

[L-04] Contracts - Unsafe ERC20 transferFrom() and transfer() Method Used in MembershipPaymentManager Contract

Description

The `MembershipPaymentManager` contract uses the standard `transferFrom` and `transfer` methods to transfer ERC20 tokens from users to the contract.

While the code correctly checks the return value with a boolean variable, this approach is still potentially unsafe. Some ERC20 tokens don't properly implement the ERC20 standard - they might not return a boolean value, might return false without reverting, or might have other non-standard behaviors. Though USDC and USDC.e generally follow the standard correctly, using direct `transferFrom` calls without additional safety checks can lead to silent failures or unexpected behavior, especially if the contract is ever adapted for other tokens.

Recommendation

Use OpenZeppelin's `SafeERC20` library which provides safe versions of ERC20 operations.

Status

Resolved

[L-05] Contracts - Missing Input Validation for Tier and Amount Parameters in Payment Functions

Description

The payment functions fail to validate the tier parameter against `MAX_TIER` and don't ensure the amount is greater than zero. This allows users to purchase non-existent tiers or specify zero amounts, resulting in negligible payments (1 wei) for seemingly legitimate membership purchases. This undermines the payment verification system and allows manipulation of the referral system.

Recommendation

Add explicit validation at the beginning of both `processPayment` and `processERC20Payment` functions to ensure `tier <= MAX_TIER` and `amount > 0`, reverting the transaction if either condition fails.

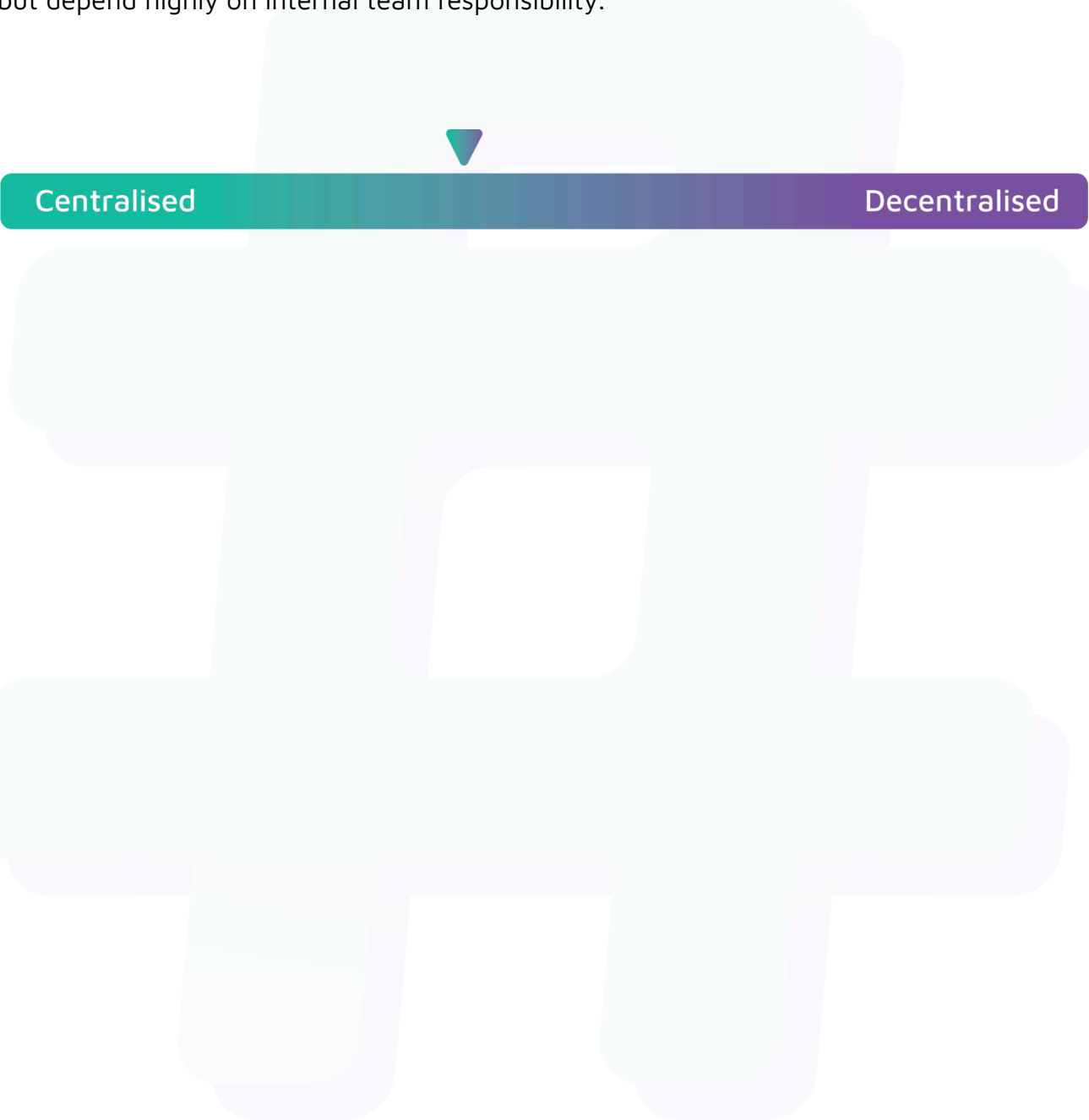
Status

Resolved

Centralisation

The Defi Bull World project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Defi Bull World project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.